Spinning Up Modern: Technical Modernization Report

Kiet Huynh

April 2026

1 Core Architectural Evolution

This report details the technical implementation of the modernization of OpenAI's Spinning Up repository. The primary goal was to transition from legacy, unoptimized software to a production-grade, GPU-accelerated research environment using the latest available library versions.

1.1 Gymnasium Migration and Environment Standardization

The project successfully transitioned from `gym` to `gymnasium` (v1.0+).

Table 1: API and Environment Standardization

Feature	Implementation Detail	Version/API
Step Logic	Decomposed Terminal/Truncated signals	`obs, rew, term, trunc, info`
Reset Logic	Information dictionary support	`obs, info`
CartPole	Standardized on stable version	`CartPole-v1`
HalfCheetah	Standardized on MuJoCo free release	`HalfCheetah-v5`

2 Deep Learning Backends

2.1 PyTorch 2.1.2: Graph Mode and GPU Coordination

PyTorch implementations have been optimized for modern hardware and software standards:

- **Kernel Fusion:** Integrated `torch.compile()` for JIT-optimized execution.
- **Memory Management:** Fixed host-device transfer bottlenecks by standardizing on `.detach().cpu().numpy()` for all environment interactions.

- **Type Safety:** Implemented comprehensive Python type hints for all Actor-Critic architectures.

2.2 TensorFlow 2.15: Eager Execution Migration

A total rewrite of the TensorFlow backend removed all legacy `tf.compat.v1` dependencies.

- **Functional Models:** All algorithms now use `tf.keras.Model` subclassing for better modularity.
- **Automatic Differentiation:** Replaced static graphs with `tf.GradientTape` orchestration.

```
with tf.GradientTape() as tape:
    loss_pi, kl, ent = compute_loss_pi(obs, act, adv, logp_old)
grads = tape.gradient(loss_pi, ac.pi.trainable_variables)
pi_optimizer.apply_gradients(zip(grads, ac.pi.trainable_variables))
```

Listing 1: Modern TF2 Policy Update Pattern

3 Hardware Acceleration: Strict Dependency Alignment

A critical technical achievement was solving the **cuDNN Version Conflict** present in modern pip-based NVIDIA distributions when combining PyTorch and TensorFlow in the same environment.

3.1 Implementation Detail: The Golden Path

TensorFlow 2.15 strictly requires `libcudnn.so.8`. However, installing the latest PyTorch versions (e.g., 2.5+) automatically pulls down `cuDNN v9` dependencies, silently breaking TensorFlow's ability to detect the GPU.

To resolve this natively without resorting to fragile runtime symlinks or `LD_LIBRARY_PATH` hacking, we established a strict dependency alignment:

1. **Version Pinning:** PyTorch is pinned to `torch==2.1.2`.
2. **Native Resolution:** This specific PyTorch release explicitly depends on `nvidia-cudnn-cu12==8.9.2.26`.
3. **Framework Harmony:** This `cuDNN` package natively provides `libcudnn.so.8`, perfectly satisfying the requirements of both `torch==2.1.2` and `tensorflow==2.15.0` simultaneously.

This ensures that the repository remains "plug-and-play" across diverse Linux environments, granting both backends full access to the GPU right out of the box.

4 Distributed Execution: MPI-GPU Synchronization

Modernized MPI support ensures correct coordination with CUDA devices:

- **Weight Sync:** Synchronized model parameters across processes using rank-0 broadcasting.
- **Gradient Aggregation:** Global gradient averaging via `mpi_avg` before optimizer updates.
- **Resource Isolation:** Configured `allow_growth` to prevent MPI ranks from conflicting on VRAM allocation.

5 Verification Results

The system passed a rigorous 100% sweep of the core algorithm suite (VPG, PPO, DDPG, TD3, SAC, TRPO) with no initialization warnings and full hardware acceleration.

6 Citation

```
@misc{SpinningUpModern2026,
    author = {Kiet Huynh},
    title = {{Spinning Up in Deep RL (Modernized)}},
    year = {2026},
    publisher = {GitHub},
    howpublished = {\url{https://github.com/tkiethuynh/spinningup}}
}
```